

Como usar python dentro do Lazarus

1. INDEX

1. [Resumo do conteúdo](#)

2. **Introdução**

1. [Objetivo.](#)
2. [Pre-requisitos.](#)
3. [Benefícios.](#)
4. [Desvantagens.](#)

3. **Conteúdo estudado.**

1. Instalar componente python4Laz
2. Exemplo de como executar um código python e capturar a saída do python para ser visualizada no Lazarus.
3. Exemplo de uso do python no modo console demonstra como informar o nome da lib python no sistema
4. Componentes Python4Laz usados para executar código python e capturar o resultado

4. **Referências globais.**

5. **Histórico.**

2. CONTEÚDO

1. **Resumo do conteúdo:**

1. Este documento descreve como instalar o pacote python4Lazarus, o objetivo desta ação, os benefícios e os malefícios de se usar python com Lazarus, bem como mostra alguns exemplos de seu uso.

2. **Introdução**

1. **Objetivo:**

1. O componente *python4laz* é um recurso que permite que o se possa aproveitar todos os projetos escrito em python no lazarus, tais como Chatgpt, aprendizado de máquina, opencv etc... visto que o lazarus tem recursos RAD que facilita e muito o desenvolvimento de um software.

2. **[BACK]**

2. **Pre-requisitos:**

1. O python deve estar instalado antes de ser instalar o python4laz;

2. Os projetos que se deseja usar no python, também deve ser instalados;
3. Lazarus deve ser instalado com fpcupdeluxe;
4. O pacote python4laz deve ser instalado com fpcupdeluxe.
5. [← BACK]

3. Benefícios:

1. Se tornar mais produtivo aproveitando tudo que existe hoje para python e tudo que existe el freepascal, bem como ter domínio do projeto através dps componentes TDataModule e TForm do Lazarus.
2. [← BACK]

4. Desvantagens.

1. O programas escritos em python são mais lentos que freepascal por ser uma linguagem interpretada;
2. Os usuários do produto pode ser deparar com erros básico de sintaxe;
3. Qualquer um tem acesso ao código fonte do python.
4. [← BACK]

3. Conteúdo estudado

1. Instalar componente python4Laz

1. Para se instalar o componente *Python4Laz* vamos utilizar o projeto [fpcupdeluxe](#) opção `/modules/python4laz`.
2. O projeto é instalado na pasta ccr onde está instalado o lazarus e o free pascal
3. [← BACK]

2. Exemplo de como executar um código python e capturar a saída do python para ser visualizada no Lazarus..

1. Ao instalar o pacote *Python4Laz*, o sistema copia para a pasta `./ccr/python4laz/demos_lazarus/Python_Console` um exemplo de como usar o projeto. O arquivo `formmain.pas` possui uma constante de nome `cPyLibraryLinux` com o nome da *lib* python instalada no sistema; a mesma deve conter o nome da *lib*.
2. Neste exemplo o nome da dll é informado na constante `cPyLibraryLinux`.
3. O python instalado no Linux mint 21.2 é:
 1. O nome do executável python instado:

```
# Nome do executável.  
/usr/bin/python3
```

2. O nome da lib python instalado:

```
Const  
  cPyLibraryLinux: string = 'libpython3.10.so.1.0';  
//Linux Mint 21.2 Cinnamon
```

3. O passo seguinte é criar um *link* para a *dll* 'libpython3.10.so.1.0' na pasta do projeto *ccr/python4laz/demos_lazarus/Python_Console/demo*;

```
ln -s /lib/python3.10/config-3.10-x86_64-linux-gnu/libpython3.10.so libpython3.10.so
```

4. Teste para saber se o python está configurado..

1. Executar o projeto *demo* da pasta
`./ccr/python4laz/demos_lazarus/Python_Console`
2. No rodapé formulário executado digite o comando python a seguir:

```
print("Alo mundo");
```

3. Nota

1. Se ao executar o texto *alo mundo* for impresso é porque o sistema está configurado.
2. .

5.

3. Exemplo de uso do python no modo console demonstra como informar o nome da lib python no sistema

1. Ao instalar o pacote *Python4Laz*, o sistema copia para a pasta `./ccr/python4laz/demos_lazarus/Python_Thread` um exemplo de como executar várias tarefas python ao mesmo tempo.
2. Para que o programa funcione é necessário selecionar a *unit Testthds* e alterar as propriedades abaixo:

```
TForm1.PythonEngine1.DllName := 'libpython3.10.so.1.0';  
TForm1.PythonEngine1.DllPath := '/lib/x86_64-linux-gnu';
```

3.

4. Componentes *Python4Laz* usados para executar código python e capturar o resultado

1. A unit *PythonEngine* contém o componente *PythonEngine* obrigatório no projeto python no Lazarus. O mesmo é usado para conectar-se a lib python instalada no sistema.

1. Exemplo de uso:

```
const
  cPyLibraryWindows: string = 'python37.dll';
  // cPyLibraryLinux: string = 'libpython3.8.so.1.
  Ubuntu 20.x
  cPyLibraryLinux: string = 'libpython3.10.so.1.0';
  Cinnamon
  cPyLibraryMac: string =
  '/Library/Frameworks/Python.framework/Versions/3.7/lib/
  cPyZipWindows: string = 'python37.zip';

procedure TfmMain.DoPy_InitEngine;
var
  S: string;
begin
  S:=
    {$ifdef windows} cPyLibraryWindows {$endif}
    {$ifdef linux} cPyLibraryLinux {$endif}
    {$ifdef darwin} cPyLibraryMac {$endif} ;

  PythonEngine.DllPath:= ExtractFileDir(S);
  PythonEngine.DllName:= ExtractFileName(S);
  PythonEngine.LoadDll;
end;
```

2. ..

4. REFERÊNCIAS GLOBAIS

1. [Site oficial para produzir este documento](#)
2. [Download do python](#)
3. [O que é cpython](#)

1. 

1. HISTÓRICO

- ☒ Criar este documento baseado no como_usar_python_dentro_do_lazarus.md ;
- ☒ Escrever tópico Objetivos;
- ☒ Escrever tópico Pre-requisitos
- ☒ Escrever tópico Benefícios
- ☒ Escrever tópico desvantagens

- ☐ Escrever tópico Conteúdo
 - ☒ Criar documento de como instalar o pacote python4laz;
 - ☒ Criar documento de como configurar o ambiente python no Lazarus.
- ☐ Escrever tópico Exemplos
- ☒ Escrever tópico Referências
- ☒ Atualizar o histórico deste documento.
- ☐ Testar este documento depois após uma semana de concluído.
 - 

